

# Multi-Tenant Mobile Antenna Placement Optimization

## A Comparative Study of OR-Tools CP-SAT and SICStus CLP(FD)

Martim Pinheiro and Adam Oprchal

Faculdade de Engenharia da Universidade do Porto, Porto, Portugal  
{up20509641, up202512712}@fe.up.pt

**Abstract.** We formulate Multi-Tenant Mobile Antenna Placement as a Constraint Satisfaction and Optimisation Problem (CSOP) over Boolean finite domains and implement it in two constraint programming solvers: Google OR-Tools CP-SAT and SICStus Prolog CLP(FD). Given a grid map with heterogeneous terrain, the model places cell towers and assigns antennas from four telecom providers with distinct Manhattan coverage radii to minimise total infrastructure cost, comprising tower construction, antenna slot fees, and soft-constraint uncoverage penalties. Both implementations share the same mathematical model and a common data pipeline. Benchmarks across grid sizes from  $4 \times 4$  to  $64 \times 64$  confirm model equivalence and reveal a large scalability gap: OR-Tools solves instances up to  $16 \times 16$  within 60 seconds, while SICStus times out beyond  $5 \times 5$ . The root causes — CDCL clause learning, LP-based lower bounds, and internal symmetry detection in CP-SAT versus pure arc-consistency propagation in CLP(FD) — are analysed in depth. Post-presentation improvements, including the first-fail (ff) labelling heuristic and extended OR-Tools benchmarks, are evaluated and discussed.

**Keywords:** Constraint Programming · Antenna Placement · OR-Tools CP-SAT · SICStus CLP(FD) · Combinatorial Optimisation · Network Planning

## 1 Introduction

Mobile network planning is a large-scale combinatorial optimisation problem: operators must decide where to build physical infrastructure and which radio services to assign to each site, subject to technical, financial, and regulatory constraints. In a *multi-tenant* setting, multiple operators co-locate equipment on shared cell towers, reducing capital expenditure while increasing site density.

This work addresses a representative instance: given a grid map with four terrain types, place cell towers and assign antennas from four providers to minimise total infrastructure cost. Coverage is a *soft constraint* modelled via a financial penalty, allowing the solver to trade tower construction against unserved cells. Two contrasting Constraint Programming (CP) platforms are implemented and compared:

- **Google OR-Tools CP-SAT** [1]: a hybrid solver combining Conflict-Driven Clause Learning (CDCL) with LP relaxations.
- **SICStus Prolog CLP(FD)** [2]: a classical finite-domain solver based on arc-consistency propagation and branch-and-bound search.

Implementing the same model in both solvers allows a direct empirical comparison of two CP paradigms and provides concrete evidence of how solver architecture affects scalability on a realistic problem.

## 2 Problem Description

### 2.1 Grid and Terrain

The search space is an  $R \times C$  grid of cells, each with one of four terrain types that determine tower buildability and cost (Table 1). Terrain is generated randomly with fixed proportions: 12% mountains (banned), 20% forest, 15% building, remainder plains.

Table 1: Terrain types and tower construction costs.

| Terrain  | Symbol | Tower cost (€) | Buildable? |
|----------|--------|----------------|------------|
| Plains   | .      | 1 000          | Yes        |
| Forest   | F      | 1 500          | Yes        |
| Building | B      | 2 500          | Yes        |
| Mountain | M      | —              | No         |

### 2.2 Telecom Providers and Coverage

Four providers (P0–P3) operate on different frequency bands with different Manhattan-radius coverage patterns (Table 2). An antenna at cell  $i$  covers all non-banned cells within Manhattan distance  $r_k$ .

Table 2: Provider frequency bands and coverage radii.

| Provider | Band                  | Radius $r_k$ | Technology     |
|----------|-----------------------|--------------|----------------|
| P0       | High-band / 5G mmWave | 0            | Own cell only  |
| P1       | Mid-band / 4G–5G      | 1            | Adjacent cells |
| P2       | Mid-band / 4G–5G      | 1            | Adjacent cells |
| P3       | Low-band / Macro LTE  | 2            | Wide area      |

P1 and P2 share radius  $r=1$ , introducing solution symmetry: swapping their placements yields an identical-cost solution. This doubles the effective search space unless broken explicitly (Section 6.4).

### 2.3 Cost Structure

Total cost has three additive components: (1) tower construction (€1 000–€2 500, terrain-dependent); (2) antenna slot fees (€100 per antenna); (3) uncoverage penalties (€800 per uncovered provider–cell pair). The penalty mechanism transforms coverage into a soft constraint: covering a remote cell is skipped when its cost exceeds the penalty savings.

## 3 Formal Model

Let  $N=R \times C$ ,  $K=4$ ,  $\mathcal{B}$  the banned cells, and  $\mathcal{C}=[0, N) \setminus \mathcal{B}$  the coverable cells.

**Decision variables:**

$$\begin{aligned} t_i &\in \{0, 1\} \quad i \in [0, N) \\ a_{k,i} &\in \{0, 1\} \\ s_{k,j} &\in \{0, 1\} \quad j \in \mathcal{C} \end{aligned}$$

$t_i = 1$  means a physical tower is erected at cell  $i$ ;  $t_i = 0$  means the cell is left empty.  $a_{k,i} = 1$  means provider  $k$  installs an antenna at cell  $i$ .  $s_{k,j} = 1$  means cell  $j$  is *not* covered by provider  $k$ , activating the financial penalty for that pair. Total variables:  $N + KN + K|\mathcal{C}|$  (308 for a  $6 \times 6$  grid).

**Coverage sources:**

$$\text{cov}(k, j) = \{i \in [0, N) \setminus \mathcal{B} \mid \|i - j\|_1 \leq r_k\}$$

$\text{cov}(k, j)$  is the set of all non-banned cells from which an antenna of provider  $k$  would reach target cell  $j$  within Manhattan distance  $r_k$ . It is precomputed before the solver runs.

**Constraints:**

$$\mathbf{C1} \quad t_i = 0 \quad \forall i \in \mathcal{B}$$

No tower may be built on a mountain cell. These cells are permanently banned from the search space.

$$\mathbf{C2} \quad a_{k,i} \leq t_i \quad \forall k, \forall i$$

An antenna can only be installed where a physical tower already exists. If no tower is built at cell  $i$ , all provider antenna variables for that cell are forced to zero.

$$\mathbf{C3} \quad \sum_{k=0}^{K-1} a_{k,i} \leq 4 \quad \forall i$$

At most four providers may share a single tower. This is the multi-tenancy capacity constraint; since  $K = 4$  in this configuration, it prevents a single tower from being over-committed if the number of providers were extended in the future.

$$\mathbf{C4} \quad \sum_{i \in \text{cov}(k,j)} a_{k,i} + s_{k,j} \geq 1 \quad \forall k, \forall j \in \mathcal{C}$$

For every (provider, cell) pair, either at least one antenna within coverage range is active, or the slack variable  $s_{k,j}$  is set to 1. When the slack is 1, the cell is left uncovered and the penalty is charged in the objective. This is the *reified* form of the coverage constraint: it allows the solver to decide whether covering a particular cell is economically worthwhile.

**Objective:**

$$\min Z = \underbrace{\sum_i c_i t_i}_{\text{tower construction}} + 100 \underbrace{\sum_{k,i} a_{k,i}}_{\text{antenna slots}} + 800 \underbrace{\sum_{k,j} s_{k,j}}_{\text{uncoverage penalties}}$$

The first term accumulates the terrain-dependent tower construction cost ( $c_i \in \{1000, 1500, 2500\}$ ) for every tower that is built. The second term charges a flat €100 fee for each antenna slot used, regardless of provider or terrain. The third term penalises every (provider, cell) pair that is left without coverage. Because all three terms are linear combinations of Boolean variables, the problem is also encodable as a Binary ILP.

## 4 Implementation

### 4.1 System Architecture

A single JSON file (`map_data.json`) serves as the shared data source for both solver pipelines (Fig. 1). This guarantees both solvers operate on exactly the same instance. A Python preprocessor (`preprocessor.py`) translates the JSON into Prolog facts (`map_data.pl`) for the SICStus solver. A benchmark runner (`benchmark.py`) orchestrates both solvers, collects timing and cost data, and writes results to CSV.

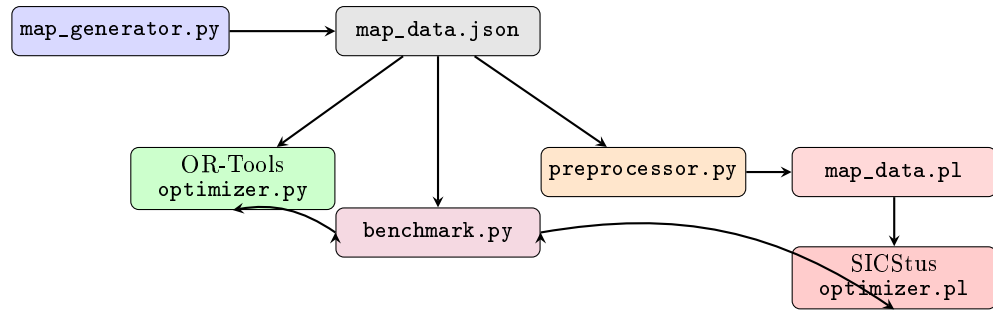


Fig. 1: System architecture: single JSON source feeds two independent solver pipelines and the shared benchmark runner.

## 4.2 OR-Tools CP-SAT

The OR-Tools implementation uses the `cp_model` Python API. All variables are `NewBoolVar` instances. Coverage sources are precomputed once before model construction. The coverage-or-penalty constraint C4 is posted as a single `model.Add(...)` call per (provider, cell) pair:

```
for (k, tgt), sources in coverage_sources.items():
    model.Add(sum(antennas[k][src] for src in sources)
              + slacks[k][tgt] >= 1)
```

The solver is configured with a wall-clock timeout and accepts both `OPTIMAL` and `FEASIBLE` statuses.

## 4.3 SICStus Prolog CLP(FD)

The SICStus implementation uses `library(clpfd)`. Variables are 0/1 domain lists: *T* (towers), *P0–P3* (antennas per provider), *S0–S3* (slacks per provider). Cost terms are built with `scalar_product/4` and summed via `#=`. The coverage constraint is expressed as:

```
extract_domain_vars(ValidSources, P_k, CoverageVars),
sum([SlackVar | CoverageVars], #>=, 1)
```

Optimisation uses branch-and-bound via `labeling/2`:

```
labeling([ff, minimize(GlobalCost)], AllVars)
```

The `ff` (first-fail) option branches on the variable with the smallest remaining domain (see Section 7.1).

# 5 Experimental Results

## 5.1 Setup

All benchmarks use a 60-second timeout per solver, seed 42 for reproducibility, and nine grid sizes from  $4 \times 4$  to  $64 \times 64$ . The benchmark runner invokes both solvers as independent subprocesses and records wall-clock time, optimal cost, and coverage percentage.

## 5.2 Results

Table 3: Benchmark results (60 s timeout, seed 42). T/O = timeout.

| Grid  | Cells | OR-Tools CP-SAT |          |       | SICStus CLP(FD) |          |
|-------|-------|-----------------|----------|-------|-----------------|----------|
|       |       | Time (ms)       | Cost (€) | Cov.% | Time (ms)       | Cost (€) |
| 4×4   | 16    | 390             | 13 600   | 82    | 371             | 13 600   |
| 5×5   | 25    | 397             | 22 500   | 84    | 14 373          | 22 500   |
| 6×6   | 36    | 418             | 31 600   | 82    | T/O             | —        |
| 7×7   | 49    | 461             | 42 400   | 82    | T/O             | —        |
| 8×8   | 64    | 495             | 55 500   | 82    | T/O             | —        |
| 12×12 | 144   | 1 377           | 122 600  | 82    | T/O             | —        |
| 16×16 | 256   | 31 112          | 218 600  | 82    | T/O             | —        |
| 32×32 | 1 024 | T/O             | —        | —     | T/O             | —        |
| 64×64 | 4 096 | T/O             | —        | —     | T/O             | —        |

## 5.3 Model Correctness

On all instances where both solvers complete (4×4 and 5×5), they produce **identical optimal costs**, confirming that both implementations encode exactly the same CSOP. Additional multi-seed validation (seeds 42, 123, 456 for 4×4–5×5) yielded six matching cost pairs across both solvers.

## 5.4 Timing and Coverage

Figure 2 plots solve time on a log scale. The gap is already visible at 5×5: OR-Tools solves in 397 ms; SICStus takes 14.4 s — a **36× difference**. From 6×6 onward SICStus consistently times out. OR-Tools scales to 16×16 (31 s) but also times out at 32×32 (1 024 cells), revealing its own ceiling.

Both solvers achieve **82–84% coverage** on all completed instances. The uncovered 16–18% are economically rational: building a tower for those cells costs more than the €800 penalty. Coverage is stable across all grid sizes, confirming that the penalty formulation generalises uniformly.

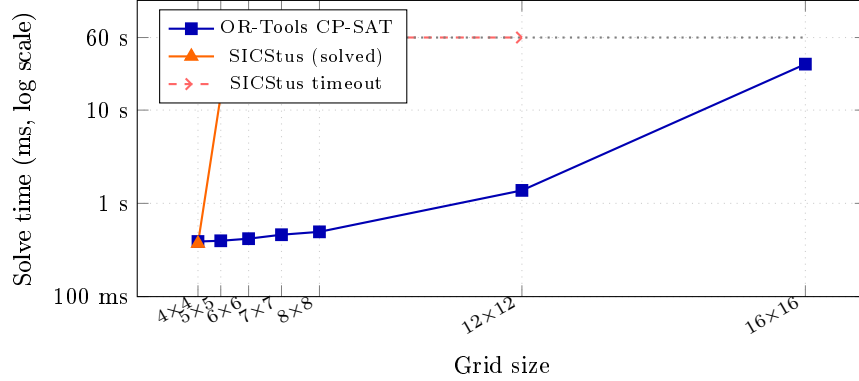


Fig. 2: Solve-time comparison (log scale). SICStus data points for 6×6 and beyond all hit the 60-second ceiling.

## 6 Performance Analysis

Four compounding factors explain the scalability gap. These are not fundamental CLP(FD) limitations — they are configuration choices, each independently addressable.

### 6.1 Naive Variable Ordering

Without heuristics, `labeling/2` assigns variables left-to-right: towers first, then antennas, then slacks. The solver commits to a complete tower layout before considering coverage, discovering conflicts late and triggering expensive deep backtracking.

### 6.2 Weak Lower Bounds

SICStus branch-and-bound prunes branches only when constraint propagation can prove cost exceeds the current best. These bounds are often loose. OR-Tools CP-SAT solves an LP relaxation at each node, providing tight fractional lower bounds that prune entire subtrees with a certificate of suboptimality.

### 6.3 Arc Consistency vs. Clause Learning

CLP(FD) uses arc-consistency propagation (AC-3/AC-4): sound and complete, but memoryless — backtracking discards all intermediate reasoning. OR-Tools CP-SAT uses CDCL [5]: on conflict, it analyses the implication graph, derives a *no-good clause*, and adds it permanently. Learned clauses prune future branches that recreate the same conflict. CLP(FD) backtracks and forgets; CP-SAT backtracks and *learns*.

## 6.4 Unhandled Solution Symmetry

Providers P1 and P2 have identical radii ( $r=1$ ). Swapping their placements yields an equal-cost solution. Without a symmetry-breaking constraint, the solver explores both symmetric halves, roughly doubling the search. The fix is a single lexicographic ordering constraint: `lex_chain([P1, P2])`. OR-Tools handles this internally; the SICStus model does not include it.

Table 4 summarises all four factors.

Table 4: Performance gap factors and targeted remedies.

| Factor                  | Impact                       | Remedy                    |
|-------------------------|------------------------------|---------------------------|
| Naive variable ordering | Late conflict detection      | <code>ff</code> heuristic |
| Weak lower bounds       | Poor subtree pruning         | LP relaxation             |
| No clause learning      | Repeated identical conflicts | CDCL                      |
| Unhandled symmetry      | $\approx 2\times$ search     | <code>lex_chain</code>    |

## 7 Implemented Improvements

### 7.1 First-Fail Heuristic

The `ff` variable ordering was added to the SICStus solver:

```
labeling([ff, minimize(GlobalCost)], AllVars)
```

Table 5 compares solve times before and after.

Table 5: SICStus solve times with and without `ff` (seed 42).

| Grid | Without <code>ff</code> |          | With <code>ff</code> |          |
|------|-------------------------|----------|----------------------|----------|
|      | Time (ms)               | Cost (€) | Time (ms)            | Cost (€) |
| 4×4  | 371                     | 13 600   | 382                  | 13 600   |
| 5×5  | 14 373                  | 22 500   | 14 785               | 22 500   |
| 6×6  | T/O                     | —        | T/O                  | —        |

The `ff` heuristic has **negligible effect**: timing differences are within run-to-run noise and 6×6 continues to time out. This is itself an instructive result: variable ordering alone cannot compensate for the absence of tight lower bounds and clause learning. Practitioners should not expect heuristic ordering to bridge a fundamental solver-architecture gap on optimisation problems of this type.



## 8 Conclusions

We presented a CSOP formulation for multi-tenant antenna placement, implemented it in two CP solvers, and evaluated it across nine grid sizes. Key findings:

1. **Model equivalence:** both solvers produce identical optimal costs on all shared-completed instances, validating both implementations.
2. **Large scalability gap:** OR-Tools solves up to  $16 \times 16$ ; SICStus times out at  $6 \times 6$  — over an order of magnitude in problem size.
3. **Architectural root causes:** the gap stems from CDCL learning, LP-based lower bounds, and internal symmetry handling in CP-SAT, none of which are present in vanilla CLP(FD) branch-and-bound.
4. **First-fail is insufficient alone:** adding `ff` to SICStus had negligible impact, confirming that lower-bound quality and learning capacity are the bottleneck, not variable ordering.
5. **OR-Tools has its own ceiling:** it times out at  $32 \times 32$  (1 024 cells), showing that city-scale deployment requires further techniques.
6. **Soft-constraint coverage works:** 82–84% coverage is achieved consistently, correctly reflecting the economic trade-off between tower cost and coverage penalty.

Future work includes symmetry breaking via `lex_chain([P1, P2])` in SICStus, spatial decomposition for larger OR-Tools instances, and grid visualisation for qualitative solution analysis.

## References

1. Google OR-Tools development team: OR-Tools CP-SAT Solver. <https://developers.google.com/optimization> (2024)
2. Swedish Institute of Computer Science: SICStus Prolog 4.x User’s Manual. <https://sicstus.sics.se/documentation.html> (2024)
3. Apt, K.R.: Principles of Constraint Programming. Cambridge University Press (2003)
4. Rossi, F., van Beek, P., Walsh, T. (eds.): Handbook of Constraint Programming. Elsevier (2006)
5. Marques-Silva, J.P., Sakallah, K.A.: GRASP: A search algorithm for propositional satisfiability. IEEE Trans. Comput. **48**(5), 506–521 (1999)